QuickFlow => Intelligent Order Processing & Scheduling Engine.

mini-version of the backend engine behind Amazon/flipkart.

→ "You system should intelligently decide when the order is placed:

1. which warehouse should fulfill it.
2. which inventory should be used.
3. which worker/process should handle it.
4. what order should be processed first
5. what if an item is out of stock
6. what if 1,000 orders arrive simultaneously
7. what if a worker/warehouse goes down
8. what if two orders try to reserve the same item
9. what if high priority order arrives
10. can orders be rescheduled
11. can we retry failed jobs
12. can we track the complete order lifecycle.

Frontend → React
Backend → node, express
Database → MongoDB
Core Engine → C++
OOP → C++ classes/interface/polymorphism
Communication → REST API / internal API
Containerization → Docker
Orchestraction → Docker compose

Authentication → JWT
Concurrency
HLD
LLD
DSA
Thread pool

1000 orders

↓

Order Gateway

↓

Priority Engine

↓

Inventory manager

↓

Scheduling Engine

↓

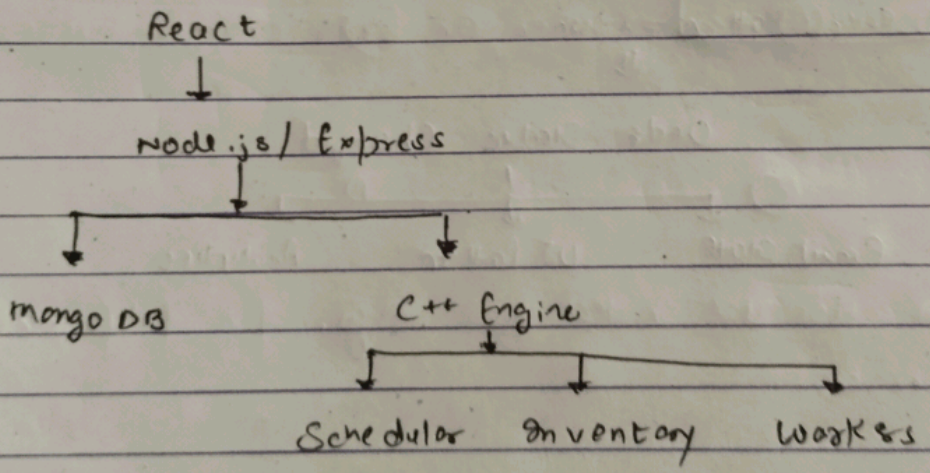
Worker Pool

↓

Fulfillment

↓

Delivery Assignment



Q → What is Quickflow :-

Ans → Quickflow ek intelligent backend engine hai jo online shopping ke orders ko recieve karta hai aur automatically decide karta hai ki order ko kis warehouse, kis worker, kis priority aur kis processing sequence main fulfill karta hai.

→ Patterns to use:

FIFO

Priority

Deadline

Shortest Job

Hybrid

Factory Pattern

→ Tasks to add:

Picking

Packing

Shipping

Restocking

Observer Pattern => When an order changes

⇓

Order Status Changed

Email Store | UI Pattern | Analytics

→ Order lifecycle:

CREATED → CONFIRMED → ALLOCATED →

PROCESSING → PACKED → SHIPPE → DELIVERED

- Invalid Transaction is to prevented.

⇓

Key feature

→ Priority queue:

⇓

Urgent orders
VIP orders
Deadline orders
Normal orders

⇓

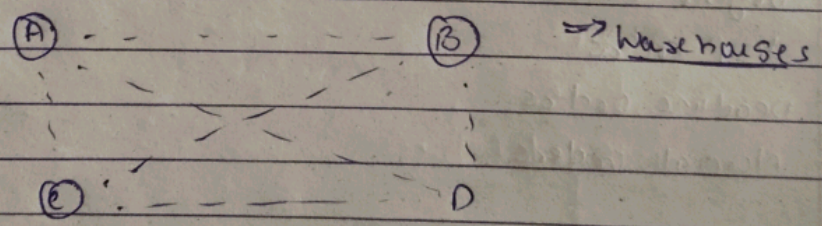
Eg: Order A → Priority 10
Order B → Priority 3
Order C → Priority 8

⇒ Processing is like:
A → C → B.

Hashmaps:
⇓

product ID → inventory
order ID → order
warehouse ID → warehouse
worker ID → worker

Graphs: Used to represent warehouses and delivery locations as:



Now, Here use shortest-path algo. for fulfilment/delivery decision.

- Dijkstra
- Concurrency.

→ On C++ demonstrate

⇓

- std:: thread
- std:: mutex
- std:: conditional_variable
- std:: atomic
- Build ~~prop~~ thread pool.

Q- How do you prevent two threads from reserving the same memory?

⇓

- mutex
- Critical section
- Atomic inventory update

Impressive

→ "What - if Simulation"

↓

Manager says "Tomorrow we expect 50,000 orders"

↓

Your system can simulate

↓

Current workers : 20

Excepted orders : 50,000

Estimated completion : 18.4 hrs

Then : Add 10 Workers

↓

Estimated completion : 12.1 hrs

(or)

Change scheduling algorithm

Priority → Deadline

⇓

Late deliveries ↓ 27%

C++ :

⇩

Inventory

Warehouse

Worker

Task

Scheduler

Strategy

State Machine

Thread Pool

Node.js

REST API
Authentication
Order API's
Admin API's
Analytics API's

React

Live Orders
Queue
Warehouse Status
Scheduling Metrics
Failed Jobs
System load

QuickFlow: a concurrent order-processing and scheduling engine using C++ OOP, priority based scheduling, thread pools, inventory locking, and shortest path fulfillment optimization, exposed through a mern Dashboard and containerization with Docks.

[illegible]

→ Real world scenario

⇩

Imagine you have 4-warehouses

W1 Delhi

W2 Mumbai — — — — — — — — W3 Hyderabad

W4 Kolkata

Every warehouse have different inventory

Eg:-

Delhi Warehouse

- iPhone → 20
- Laptop → 5
- Mouse → 50

My other have others.